CAHIER DES CHARGES PRODUIT

RS-BIZZ

Copilote financier Web pour micro-entrepreneurs africains

Document de cadrage (« Prompt Master ») décrivant la vision produit, l'expérience utilisateur, le modèle économique et l'architecture technique de l'application.

Premier marché cible : Cameroun

Version 1.0 — Août 2026

Sommaire

Partie I — Vision, contexte et expérience utilisateur

> *Sections 1 à 9 : le problème à résoudre, la promesse produit et les principes qui doivent guider chaque écran de RS-BIZZ.*

1. CONTEXTE DU PROJET

Concevoir et développer une application Web SaaS appelée :

RS-BIZZ

RS-BIZZ est destinée aux micro-entrepreneurs, commerçants, artisans, vendeurs, restaurateurs, coiffeurs, couturiers, mécaniciens, prestataires de services et petits entrepreneurs d'Afrique, avec un premier marché cible au Cameroun.

Le problème fondamental à résoudre est simple :

> *Beaucoup de micro-entrepreneurs vendent tous les jours, mais ne savent pas précisément combien ils gagnent réellement, où leur argent part, quels produits sont rentables, quels clients leur doivent de l'argent et comment améliorer leur activité.*

Une grande partie de ces entrepreneurs utilise encore :

- Un cahier
- Des feuilles volantes
- WhatsApp
- Leur mémoire
- Des calculs manuels

RS-BIZZ doit donc remplacer progressivement le cahier par un copilote financier extrêmement simple, sans donner à l'utilisateur l'impression d'utiliser un logiciel de comptabilité complexe.

2. NATURE DU PRODUIT

RS-BIZZ est avant tout une :

> *APPLICATION WEB RESPONSIVE*

Elle doit fonctionner parfaitement sur :

Ordinateur

- Desktop
- Laptop
- Écrans larges

Smartphone

- Android
- iPhone
- Petits écrans

Tablette

- Portrait
- Paysage

L'utilisateur doit pouvoir accéder à RS-BIZZ directement depuis son navigateur.

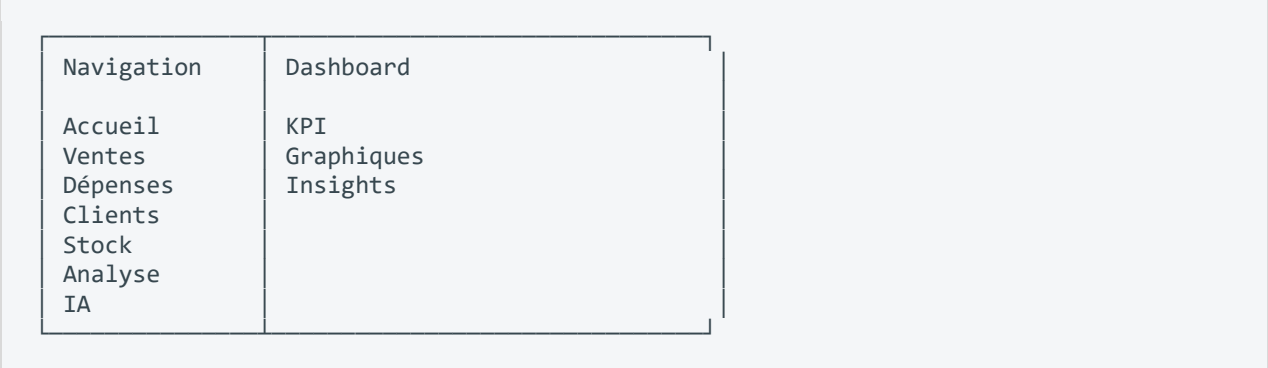
Prévoir une architecture responsive-first, mais ne pas concevoir uniquement une interface mobile agrandie. L'interface doit être pensée intelligemment pour adapter la hiérarchie des informations selon la taille de l'écran.

3. RESPONSIVE DESIGN

Créer trois expériences cohérentes :

Desktop

Utiliser pleinement l'espace disponible :



Tablette

- Sidebar compacte
- Cartes réorganisées
- Graphiques redimensionnés
- Tableaux scrollables si nécessaire

Smartphone

- Navigation inférieure
- Cartes empilées
- Boutons larges
- Tableaux transformés en cartes

- Formulaires simplifiés
- Actions principales toujours accessibles

4. VISION PRODUIT

Ne pas construire simplement :

> *« Une application de comptabilité »*

Construire plutôt :

> *RS-BIZZ : un copilote financier qui transforme les opérations quotidiennes d'un petit commerce en décisions simples et compréhensibles.*

L'utilisateur doit pouvoir ouvrir RS-BIZZ et comprendre immédiatement :

Aujourd'hui

- Combien j'ai vendu ?
- Combien j'ai dépensé ?
- Combien j'ai gagné ?
- Qui me doit de l'argent ?
- Combien mon activité possède approximativement ?
- Est-ce que mon activité va mieux ou moins bien qu'avant ?

Cette semaine

- Mes ventes augmentent-elles ?
- Mes dépenses augmentent-elles ?
- Quels produits fonctionnent le mieux ?
- Quelle est ma marge ?
- Où est-ce que je perds de l'argent ?

Ce mois-ci

- Ai-je réellement progressé ?
- Quel est mon bénéfice estimé ?
- Quelles dépenses dois-je surveiller ?
- Que dois-je améliorer ?

5. PRINCIPE UX ABSOLU

RS-BIZZ doit être compréhensible par une personne qui n'a jamais étudié la comptabilité.

Interdiction de construire une interface remplie de :

- Débit
- Crédit
- Actif
- Passif
- Bilan
- Compte de résultat
- Amortissement
- EBITDA
- Ratios complexes
- Jargon comptable

Ces notions peuvent exister dans le backend et dans les analyses avancées, mais elles ne doivent pas être imposées à l'utilisateur débutant.

Exemple

Ne pas afficher :

> *Marge brute : 32 %*

Afficher :

> *Tu gagnes environ 320 FCFA sur chaque 1 000 FCFA de vente.*

Puis éventuellement :

> *Marge : 32 %*

L'application doit toujours privilégier :

> ➜ **Langage humain → explication → donnée technique optionnelle.**

6. PRINCIPES UX

Respecter les règles suivantes :

Règle 1 — Une action = une intention

L'utilisateur doit pouvoir enregistrer rapidement :

- Une vente
- Une dépense
- Une dette client
- Un paiement reçu
- Un achat fournisseur

Règle 2 — Minimum de saisie

Ne demander que les informations réellement nécessaires.

Règle 3 — Tout doit être explicable

Chaque indicateur financier important doit pouvoir être accompagné d'un « Pourquoi ? ».

Exemple :

> *Bénéfice estimé : 82 500 FCFA*

Bouton :

> *Pourquoi ?*

L'application explique :

> *Tu as enregistré 250 000 FCFA de ventes et 167 500 FCFA de dépenses/coûts.*

Règle 4 — Feedback immédiat

Après une vente :

> *Vente enregistrée*
> *+5 000 FCFA*

Puis :

> *Ton chiffre d'affaires du jour est maintenant de 47 500 FCFA.*

Règle 5 — L'application doit apprendre progressivement

Un nouvel utilisateur ne doit pas voir 20 fonctionnalités.

Commencer avec :

① **Vendre → Dépenser → Voir mon résultat**

Puis débloquer progressivement :

② **Clients → Stock → Analyses → IA → Rapports → Fonctionnalités Premium**

7. POSITIONNEMENT

Positionnement marketing :

> *« Ton activité dans ta poche. »*

Alternative :

> *« Vends. Suis ton argent. Fais grandir ton activité. »*

RS-BIZZ doit être perçue comme :

- Simple
- Utile
- Rassurante
- Africaine
- Moderne
- Accessible
- Professionnelle
- Non intimidante

8. CIBLE UTILISATEUR

Créer plusieurs profils :

Persona	Description	Besoins clés
1 — Petit commerçant	Vend des produits dans une boutique ou un marché.	Ventes, dépenses, stock, bénéfice
2 — Vendeur WhatsApp / réseaux sociaux	Vend via WhatsApp, Facebook, Instagram.	Commandes, clients, paiements, bénéfice par vente
3 — Prestataire	Coiffeur, mécanicien, couturier, photographe, informaticien.	Prestations, dépenses, clients, revenus

Persona	Description	Besoins clés
4 — Restaurateur	Gère un restaurant ou un point de restauration.	Ventes quotidiennes, matières premières, dépenses, marge, produits les plus rentables

9. ONBOARDING

Créer un onboarding extrêmement simple.

Étape 1

> *Bienvenue sur RS-BIZZ* 👋
> *Nous allons t'aider à mieux comprendre ton activité.*

CTA : Commencer

Étape 2

Question : Que fais-tu ?

Options visuelles :

- 🛒 Commerce
- 🍔 Restaurant
- ✂️ Beauté / coiffure
- 🔧 Artisanat
- 💻 Services
- 📦 Vente en ligne
- Autre

Étape 3

Comment veux-tu suivre ton activité ?

- Simple
- Avec stock
- Avec clients à crédit
- Suivi complet

Étape 4

Combien vends-tu environ par jour ?

Proposer des fourchettes plutôt qu'un champ obligatoire.

Étape 5

Créer automatiquement le tableau de bord adapté au secteur.

Partie II — Frontend, navigation et parcours cœur de métier

> *Sections 10 à 21 : la stack front-end, la navigation responsive, le tableau de bord et les parcours de saisie qui composent l'usage quotidien de l'application.*

10. FRONTEND

Construire un frontend moderne, responsive et Web-first.

Stack recommandée :

- React
- TypeScript
- Next.js ou Vite selon architecture
- Tailwind CSS
- shadcn/ui ou système de composants équivalent
- Zustand ou autre state manager léger
- React Query / TanStack Query
- PWA
- Support offline-first

Architecture frontend :

```
src/
├── app/
├── components/
├── features/
│   ├── dashboard/
│   ├── sales/
│   ├── expenses/
│   ├── customers/
│   ├── inventory/
│   ├── analytics/
│   ├── ai/
│   ├── subscription/
│   └── settings/
├── hooks/
├── lib/
├── services/
├── stores/
├── types/
└── utils/
```

11. NAVIGATION WEB RESPONSIVE

Desktop

Sidebar permanente :

- 🏠 Accueil
- 💰 Ventes
- 💸 Dépenses
- 👥 Clients
- 📦 Stock
- 📊 Analyse
- 🤖 Mon copilote
- 💎 Premium
- ⚙️ Paramètres

Tablette

Sidebar compacte ou navigation adaptable.

Smartphone

Navigation principale :

- 🏠 Accueil
- ➕ Ajouter
- 📊 Analyse
- 👥 Clients
- ⚙️ Plus

Le bouton Ajouter doit être extrêmement visible. Lorsqu'on appuie dessus :

> *Qu'est-ce que tu veux enregistrer ?*

- 💰 Une vente
- 💸 Une dépense
- 👤 Une dette client
- 💵 Un paiement reçu
- 📦 Un achat de stock

12. DASHBOARD

Le dashboard est la page la plus importante.

Afficher en premier :

> *Bonjour*
> *Voici comment va ton activité aujourd'hui.*

Cartes principales :

Carte	Valeur affichée
Ventes aujourd'hui	47 500 FCFA (+12 % vs hier)
Bénéfice estimé	18 200 FCFA
Dépenses	29 300 FCFA
Argent que les clients te doivent	65 000 FCFA

Insight du jour — créer un bloc intelligent :

> *À savoir*
> *Tes ventes ont augmenté de 18 % cette semaine, mais tes dépenses ont augmenté de 27 %. Ton bénéfice progresse donc moins rapidement.*

CTA : Comprendre

13. SAISIE D'UNE VENTE

Titre : Nouvelle vente

Champs :

Champ	Détail
Produit / service	Sélection ou création rapide
Prix	Exemple : 5 000 FCFA
Quantité	1
Mode de paiement	Espèces, Mobile Money, Virement, Crédit
Client	Optionnel

CTA : Enregistrer la vente

Après validation :

> *Vente enregistrée*
> *Tu viens d'ajouter 5 000 FCFA à tes ventes.*

14. SAISIE D'UNE DÉPENSE

Titre : Nouvelle dépense

Catégories simples :

- Marchandise
- Transport
- Loyer
- Électricité
- Téléphone / Internet
- Salaires
- Livraison
- Autre

Champs additionnels : montant, description optionnelle.

CTA : Enregistrer

15. GESTION DES CLIENTS

Créer un CRM ultra-simple. Chaque client possède :

- Nom
- Téléphone
- Historique d'achat
- Montant total acheté
- Montant restant à payer
- Dernière transaction

Affichage :

> *Paul*
> *Doit : 25 000 FCFA*

CTA : Enregistrer un paiement

16. GESTION DU CRÉDIT

Le système doit clairement distinguer :

Statut	Montant
Argent reçu	25 000 FCFA

Statut	Montant
Argent attendu	15 000 FCFA

Ne jamais mélanger automatiquement les deux.

> *Attention : 15 000 FCFA de ventes ne sont pas encore encaissés.*

17. STOCK

Module activable selon le profil. Pour chaque produit :

- Nom
- Quantité disponible
- Prix d'achat
- Prix de vente
- Marge estimée
- Seuil d'alerte

Exemple :

> *Coca-Cola*
> *Stock : 12 unités*

> ⚠ *Ton stock devient faible.*

18. ANALYTICS

L'application doit transformer les données en visualisations très simples.

Graphique	Contenu
1 — Mes ventes	Jour / semaine / mois
2 — Mes dépenses	Répartition par catégories
3 — Mon bénéfice	Évolution temporelle
4 — Mes produits	Top produits par chiffre d'affaires
5 — Mes produits les plus rentables	Classement par marge

Les graphiques doivent rester parfaitement lisibles sur smartphone, tablette et desktop.

19. IA — COPILOTE FINANCIER

Créer un véritable module : 🤖 Mon copilote.

L'utilisateur peut poser :

> *« Est-ce que je gagne bien ? »*

Réponse :

> *Tes ventes ont augmenté de 21 % ce mois-ci. Cependant, tes dépenses ont augmenté de 29 %. Ton bénéfice estimé progresse seulement de 8 %.*

Question :

> *« Que dois-je améliorer ? »*

Réponse :

Je vois trois points :

1. Tes dépenses de transport ont augmenté de 18 %.
2. Le produit X possède une faible marge.
3. Tes ventes du samedi sont particulièrement fortes.

> *Je te conseille de mieux contrôler les achats du produit X et de renforcer ton stock avant le week-end.*

⚠ **L'IA ne doit jamais inventer de données : elle doit exclusivement utiliser les données disponibles dans la base.**

20. MOTEUR DE RECOMMANDATIONS

Créer un système de règles + IA. Exemples :

Type	Exemple
Détection d'anomalie	⚠ Tes dépenses sont 40 % supérieures à ta moyenne habituelle.
Opportunité	📈 Tes ventes du samedi sont 35 % supérieures aux autres jours.
Stock	📦 Le produit X devrait bientôt être en rupture.

Type	Exemple
Rentabilité	💰 Le produit Y génère la meilleure marge.
Crédit	👥 4 clients représentent 62 % de ton argent actuellement non encaissé.

21. RAPPORT HEBDOMADAIRE

Chaque semaine :

> *Résumé de ton activité*

Afficher :

- Ventes
- Dépenses
- Bénéfice
- Évolution
- Produits performants
- Clients débiteurs
- Recommandations

Avec une phrase :

> *« Cette semaine, ton activité va mieux qu'en début de mois. »*

ou :

> *« Tes ventes progressent, mais tes dépenses augmentent trop rapidement. »*

Partie III — Modèle économique et monétisation

> *Sections 22 à 26 : la structure Free / Premium, les fonctionnalités monétisées et la logique du paywall.*

22. PREMIUM — MODÈLE ÉCONOMIQUE

L'application doit être conçue comme un actif SaaS monétisable, pas comme une application gratuite sans stratégie commerciale. Créer plusieurs niveaux.

FREE

Fonctions essentielles :

- Ventes
- Dépenses
- Dashboard
- Historique limité
- Statistiques simples

Objectif :

> 🎯 **Permettre à l'utilisateur de comprendre immédiatement la valeur du produit.**

23. PREMIUM

Fonctions Premium :

- 📊 Analytics avancés — tendances, comparaison, marges, rentabilité, prévisions
- 🤖 Copilote IA — analyses, recommandations, questions en langage naturel, détection d'anomalies
- 📦 Gestion avancée du stock
- 👥 CRM clients
- 📄 Rapports
- ☁️ Sauvegarde cloud avancée
- 👤 Multi-utilisateurs
- 🏪 Multi-boutiques
- 📥 Export Excel / PDF

24. PLANS PREMIUM

Prévoir une architecture permettant :

Plan	Description
Gratuit	0 FCFA
Pro	Prix configurable dynamiquement
Business	Pour entreprises ayant plusieurs utilisateurs ou plusieurs points de vente

> ⚠ **Ne jamais coder les prix directement dans le frontend : les plans doivent être récupérés depuis le backend.**

25. ABONNEMENTS

Prévoir les entités suivantes :

```
subscriptions
plans
subscription_events
payments
invoices
entitlements
```

Le backend doit déterminer précisément :

> *Quelles fonctionnalités cet utilisateur a-t-il le droit d'utiliser ?*

> ⚠ **Le frontend ne doit jamais être la source de vérité pour les permissions Premium.**

26. PAYWALL INTELLIGENT

Ne pas afficher :

> *« Cette fonctionnalité est payante. »*

Afficher plutôt :

> *Découvre où tu gagnes réellement le plus d'argent.*
> *L'analyse avancée peut identifier tes produits les plus rentables et les dépenses qui réduisent ton bénéfice.*

CTA : Passer à Pro

→ **Le paywall doit vendre la valeur, pas simplement bloquer une fonctionnalité.**

Partie IV — Backend, données, API et sécurité

> *Sections 27 à 35 : l'architecture serveur, le modèle de données, l'isolation multi-tenant, les endpoints et la sécurité, ainsi que le pilotage du produit par la donnée.*

27. BACKEND

Construire un backend robuste, sécurisé et scalable.

Stack recommandée :

- Node.js
- TypeScript
- NestJS ou architecture Express/Fastify structurée
- PostgreSQL
- Prisma ORM
- Redis si nécessaire
- API REST ou GraphQL
- JWT + refresh tokens
- OAuth éventuellement
- Système RBAC
- Validation Zod / class-validator
- Logs structurés
- Monitoring

Architecture :

```
backend/
├── modules/
│   ├── auth/
│   ├── users/
│   ├── businesses/
│   ├── sales/
│   ├── expenses/
│   ├── customers/
│   ├── inventory/
│   ├── analytics/
│   ├── ai/
│   ├── subscriptions/
│   ├── payments/
│   ├── notifications/
│   └── admin/
├── database/
├── middleware/
├── guards/
├── services/
├── jobs/
├── utils/
└── config/
```

28. MODÈLE DE DONNÉES

Créer au minimum les entités suivantes :

```
User
Business
BusinessMember
BusinessSettings

Sale
SaleItem

Expense
ExpenseCategory

Customer
CustomerPayment
CustomerDebt

Product
InventoryMovement

DashboardSnapshot
AnalyticsEvent

AIConversation
AIMessage
AIInsight

Plan
Subscription
Payment
Invoice
Entitlement

Notification
AuditLog
```

Relations correctement définies avec Prisma.

29. SÉPARATION DES DONNÉES

Chaque entreprise doit posséder un espace isolé. Aucune donnée d'une entreprise ne doit pouvoir être consultée par une autre.

Utiliser systématiquement :

```
user_id
business_id
```

dans les ressources concernées. Toutes les requêtes backend doivent vérifier le contexte d'organisation.

30. API

Créer des endpoints propres :

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh

GET  /api/dashboard
GET  /api/dashboard/summary

POST /api/sales
GET  /api/sales
GET  /api/sales/:id

POST /api/expenses
GET  /api/expenses

GET  /api/customers
POST /api/customers
POST /api/customers/:id/payment

GET   /api/products
POST  /api/products
PATCH /api/products/:id

GET /api/analytics
GET /api/analytics/profit
GET /api/analytics/products

POST /api/ai/chat
GET  /api/ai/insights

GET  /api/plans
GET  /api/subscription
POST /api/subscription/checkout

GET /api/notifications
```

31. SÉCURITÉ

La sécurité doit être pensée dès la conception. Implémenter :

- Hash sécurisé des mots de passe
- JWT sécurisé
- Refresh tokens
- Rate limiting
- Validation stricte des entrées
- Protection contre injection

- Contrôle d'accès
- Isolation multi-tenant
- Audit logs
- Chiffrement des données sensibles
- Secrets uniquement côté serveur
- Aucune clé API IA exposée au frontend

32. OFFLINE-FIRST

C'est une fonctionnalité stratégique pour le marché africain. Même si RS-BIZZ est une application Web, elle doit pouvoir fonctionner lorsque la connexion Internet est faible ou momentanément absente.

Prévoir une architecture PWA avec :

```
Local database
      ↓
Sync queue
      ↓
Conflict resolution
      ↓
Server synchronization
```

Exemple — utilisateur :

> *Enregistre une vente de 5 000 FCFA.*

Même hors ligne :

> ☑ *Vente enregistrée.*

Lorsque la connexion revient : synchronisation automatique.

33. NOTIFICATIONS

Créer des notifications utiles, pas du spam. Exemples :

- 📦 Ton stock de riz devient faible.
- 👤 Paul doit encore 25 000 FCFA.
- 📈 Tes ventes ont augmenté cette semaine.
- ⚠ Tes dépenses sont inhabituelles aujourd'hui.
- 🤖 Ton rapport hebdomadaire est disponible.

34. ADMIN DASHBOARD

Créer un véritable back-office : /admin

Dashboard :

- Utilisateurs
- Entreprises
- Utilisateurs actifs
- Nouveaux utilisateurs
- Conversion Free → Premium
- MRR
- ARR
- Churn
- Abonnements
- Revenus
- Usage IA
- Coût IA
- Fonctionnalités les plus utilisées
- Erreurs
- Activité

35. ANALYTICS BUSINESS

Le produit lui-même doit être piloté par les données. Suivre :

Dimension	Indicateurs suivis
Acquisition	Inscription, source, campagne, referral
Activation	Première vente, première dépense, première semaine active
Engagement	Sessions, ventes enregistrées, fréquence d'utilisation
Conversion	Free → Premium, essai → Premium
Rétention	J1, J7, J30
Revenue	MRR, ARR, ARPU, LTV, churn

Partie V — Design, accessibilité et internationalisation

> *Sections 36 à 39 : la stratégie de rétention, l'identité visuelle, l'accessibilité et le support multi-devises.*

36. STRATÉGIE DE RÉTENTION

Le produit doit créer une habitude.

Fréquence	Message
Chaque jour	« Voici comment va ton activité aujourd'hui. »
Chaque semaine	« Voici ce qui a changé. »
Chaque mois	« Voici ce que tu dois améliorer. »

L'objectif est que l'utilisateur considère RS-BIZZ comme son tableau de bord quotidien.

37. DESIGN SYSTEM

Créer une identité visuelle :

- Moderne
- Rassurante
- Professionnelle
- Accessible
- Mobile-first
- Parfaitement responsive

Éviter l'apparence d'un logiciel bancaire compliqué. Utiliser :

- Cartes
- Gros chiffres
- Icônes explicites
- Couleurs sémantiques
- Graphiques simples
- Boutons larges
- Espaces généreux

Couleurs sémantiques

Couleur	Signification
Vert	Positif / bénéfice
Rouge	Problème / perte

Couleur	Signification
Orange	Attention
Bleu	Information / principal

⚠ **Ne jamais dépendre uniquement de la couleur : utiliser aussi le texte et des icônes.**

38. ACCESSIBILITÉ

Prévoir :

- Textes lisibles
- Contraste élevé
- Boutons suffisamment grands
- Langage simple
- Icônes accompagnées de texte
- Feedback visuel
- Messages d'erreur compréhensibles

39. INTERNATIONALISATION

Architecture prête pour :

- Français
- Anglais

Devise initiale : XAF / FCFA

Architecture prête à supporter ultérieurement :

- XOF
- GHS
- NGN
- KES
- Autres monnaies africaines

⚠ **Ne pas coder le FCFA de manière rigide dans toute l'application.**

Partie VI — Architecture financière et intelligence artificielle

> *Sections 40 à 44 : les définitions financières de référence, les garde-fous appliqués à l'IA et la trajectoire d'évolution vers un écosystème SaaS complet.*

40. ARCHITECTURE FINANCIÈRE

Attention à ne pas confondre :

Notion	Définition
Chiffre d'affaires	Argent généré par les ventes
Encaissement	Argent effectivement reçu
Dépenses	Argent sorti
Bénéfice estimé	Revenus – coûts/dépenses selon les règles définies

L'application doit expliquer ces différences avec un langage accessible.

41. RÈGLE CRITIQUE SUR LES DONNÉES

⛔ **Aucune IA ne doit modifier silencieusement les données financières.**

L'IA peut :

- Analyser
- Expliquer
- Recommander
- Détecter

Mais une modification financière doit toujours nécessiter une action explicite de l'utilisateur.

42. ARCHITECTURE IA

Créer un service IA séparé :

```
AI Gateway
      ↓
Context Builder
      ↓
Financial Data Retrieval
      ↓
Validation
      ↓
LLM
      ↓
```

```
Response Validator
      ↓
User
```

Le modèle IA ne doit pas recevoir aveuglément toute la base : construire un contexte financier structuré. Exemple :

```
{
  "period": "2026-08",
  "sales": 1250000,
  "expenses": 780000,
  "estimated_profit": 470000,
  "top_products": [],
  "debts": 125000
}
```

43. MONÉTISATION IA

Limiter les coûts. Prévoir :

- Quota mensuel IA
- Modèles différents selon le niveau
- Cache des analyses
- Génération de rapports périodiques
- Limitation des conversations
- Monitoring du coût par utilisateur

> ⚠ **Ne jamais permettre qu'un utilisateur Premium génère un coût IA supérieur à sa valeur économique sans contrôle.**

44. ARCHITECTURE SAAS

Le système doit être conçu pour devenir progressivement :

```
Application Web
      ↓
SaaS
      ↓
Multi-business
      ↓
Multi-utilisateurs
      ↓
Multi-boutiques
      ↓
API
      ↓
Écosystème financier
```

Ne pas surarchitecturer le MVP, mais ne pas créer non plus une architecture impossible à faire évoluer.

Partie VII — Feuille de route, qualité et objectifs business

> *Sections 45 à 50 : les versions successives du produit, les règles de développement, la stratégie de test et l'ambition business finale.*

45. ROADMAP

Version	Contenu
V1 — MVP	Inscription, profil entreprise, ventes, dépenses, dashboard, bénéfice estimé, historique, offline, synchronisation
V1.5	Clients, crédits, stock, rapports, notifications
V2	IA, recommandations, analytics avancés, Premium
V3	Multi-utilisateurs, multi-boutiques, paiements, API, intégrations
V4	Financement, assurance, fournisseurs, marketplace, services B2B, partenaires financiers

> ℹ **Les fonctionnalités de la V4 ne doivent pas être développées immédiatement : elles représentent uniquement la trajectoire potentielle du produit.**

46. RÈGLES DE DÉVELOPPEMENT

Le code doit être :

- TypeScript strict
- Modulaire
- Documenté
- Testable
- Maintenable
- Sécurisé
- Scalable

Interdiction :

- De mettre toute la logique dans un seul fichier
- De hardcoder les plans Premium
- De hardcoder les permissions
- D'exposer les secrets
- De faire confiance au frontend

- De dupliquer la logique financière
- De mélanger logique métier et composants UI

47. TESTS

Créer :

Unit tests

Pour :

- Calcul bénéfice
- Calcul marge
- Gestion crédits
- Stock
- Permissions Premium

Integration tests

Pour :

- Authentification
- Ventes
- Dépenses
- Synchronisation
- Abonnements

E2E — scénario principal

```
Créer compte
      ↓
Créer entreprise
      ↓
Enregistrer vente
      ↓
Enregistrer dépense
      ↓
Voir bénéfice
      ↓
Ajouter client
      ↓
Créer crédit
      ↓
Enregistrer paiement
      ↓
Consulter analyse
      ↓
Passer Premium
      ↓
Utiliser IA
```

Tester obligatoirement le comportement de l'interface sur :

- Desktop
- Tablette portrait
- Tablette paysage
- Smartphone portrait
- Smartphone paysage

48. CRITÈRES DE RÉUSSITE UX

Avant de considérer RS-BIZZ terminé, faire tester l'application à des personnes qui n'ont aucune connaissance comptable. Elles doivent pouvoir répondre sans aide :

4. Combien ai-je vendu aujourd'hui ?
5. Combien ai-je dépensé ?
6. Combien ai-je gagné ?
7. Qui me doit de l'argent ?
8. Quel produit me rapporte le plus ?
9. Est-ce que mon activité progresse ?
10. Pourquoi mon bénéfice a-t-il changé ?

⚠ **Si elles ne comprennent pas ces réponses rapidement, l'interface doit être simplifiée.**

49. OBJECTIF BUSINESS FINAL

Ne pas construire une application dont le seul objectif est :

> *« Enregistrer des ventes »*

Construire un produit dont la valeur augmente avec le temps. Plus l'utilisateur utilise RS-BIZZ :

- Plus ses données sont structurées
- Plus les analyses deviennent pertinentes
- Plus les recommandations deviennent utiles
- Plus il dépend positivement du produit
- Plus le Premium devient intéressant

Le véritable actif de l'entreprise est donc :

🏆 **Produit + utilisateurs + données structurées + moteur analytique + IA + marque + rétention + revenus récurrents.**

50. OBJECTIF FINAL

Créer une application Web SaaS responsive que le micro-entrepreneur peut ouvrir chaque matin depuis son ordinateur, sa tablette ou son smartphone et comprendre en moins de 10 secondes :

> *« Voilà comment va mon activité. »*

Puis lui permettre de passer progressivement de :

> *« Je vends et je regarde mon argent »*

à :

> *« Je comprends mon activité et je prends de meilleures décisions. »*

L'expérience doit être extrêmement simple côté utilisateur, mais reposer sur une architecture technique, financière, analytique et SaaS solide côté système.

➜ **Ne jamais privilégier la complexité technique au détriment de la simplicité utilisateur. Le produit doit cacher la complexité derrière une expérience extrêmement simple.**